Ders 10 Çalışma Özeti

Ders 10 Çalışma Özeti

Genel Konular

- Ders büyük olasılıkla 3 Mayıs 2023 Çarşamba günü, vizeden sonraki ilk derste online yapıldı.
- Yeni bölüm: yazılım kalitesi ve test.
- İşlenen konular:
  - Kodlama aşamasında kalite: programlama dili seçimi, açıklama satırları, anlamlı değişken adları.
  - Kalite faktörleri, ISO standardı ve McCall modeli.
  - Kaliteyi sağlama ve hataları erken bulma.
  - Gözden geçirme (review), inceleme (inspection) ve test.
  - Test stratejisi: birim testi, bütünleştirme testi (big bang, yukarıdan aşağı, aşağıdan yukarı, sandviç), sistem testi (güvenlik, stres, performans), kabul testi (alfa ve beta).
  - Test türleri: saydam kutu (temel yol testi, döngü testi), kara kutu (eşdeğer bölümleme, sınır değer analizi), gri kutu.
  - Doğrulama (verification) ve geçerleme (validation).
  - Test ile hata ayıklama (debugging) farkı.
- Not dağılımı ve final sınavının biçimi.

Hocanın Özellikle Vurguladığı Kısımlar

- **Testin tek amacı hatayı bulmaktır.** Hatayı düzeltmek testin işi değildir. Bulunan hataları ayıklamak ve düzeltmek hata ayıklama (debugging) işidir.
- **Hata ne kadar geç bulunursa o kadar pahalıya mal olur.** İlk aşamaları hızlı geçip hemen kodlamaya başlamak, sonradan daha zor bulunan hatalara yol açar. Hoca bu grafiği her zaman akılda tutmayı önerdi.
- **Kara kutu ile saydam kutu arasında seçim yapılmaz.** İkisi birlikte uygulanır.
- **Saydam kutu testinin hedefi:** Bir modüldeki bütün bağımsız yolların en az bir kez çalıştırılması.
- **Sınır değer analizinin mantığı:** Hata varsa çoğunlukla sınırlarda ortaya çıkar.
- **Stres testi gibi testlerde test otomasyonu gerekir.** 199 kullanıcının aynı anda bağlandığı durumu elle denemek gerçekçi değildir.
- **Doğrulama ve geçerleme farkı:** Doğrulama “işi doğru yapıyor muyuz?”, geçerleme “doğru işi mi yapıyoruz?” sorusudur. Kabul testi tamamen geçerlemedir.
- McCabe ölçütü ve temel yol sayısının hesabı, yazılım ölçütleri dersinde işlenecek.

Kısa Tekrar Notları

- **Programlama dili seçimi:** gereksinim analizinde yapılır. Ölçütler: uygulama alanı, algoritmanın karmaşıklığı, geliştirme ortamı, çalışanların bilgi düzeyi, proje koşulları. Sistemin farklı yerlerinde farklı diller kullanılabilir.
- **Kalite faktörleri:** doğruluk, güvenilirlik, verimlilik, güvenlik, kullanışlılık, hata bulma kolaylığı, esneklik, test edilebilirlik, taşınabilirlik, yeniden kullanılabilirlik, birlikte çalışabilirlik.
- **McCall modeli:** Faktörler doğrudan ölçülmez. Her faktör, ölçülebilir metriklerin ağırlıklı toplamıyla tahmin edilir.

- **Test stratejisi (V yapısı):** kodlama ↔ birim testi, tasarım ↔ bütünleştirme testi, sistem gereksinimleri ↔ sistem testi, kullanıcı gereksinimleri ↔ kabul testi.
- **Birim testi:** Kodu görerek, ağırlıklı saydam kutu testiyle yapılır. Modülün arayüzü, veri yapıları, kontrol yapıları, yolları, hata yolları ve sınırları test edilir.
- **Bütünleştirme testi:** Modüller arası arayüz ve ilişkiler test edilir. Türleri: big bang, artımlı (yukarıdan aşağı, aşağıdan yukarı), sandviç.
- **Sistem testi:** Yazılımın donanım, işletim sistemi ve veritabanıyla birlikte çalışması test edilir. Başlıcaları güvenlik, stres (dayanıklılık) ve performans testleridir.
- **Kabul testi:** Müşteriyle birlikte, SRS'e göre ve ağırlıklı kara kutu testiyle yapılır.
- **Alfa testi:** Geliştirici ekibin denetiminde, kullanıcıya benzer bir grup tarafından yapılır.
- **Beta testi:** Gerçek ya da potansiyel kullanıcılar tarafından yapılır.
- **Gri kutu testi:** Test verisi bir yaklaşımla hazırlanıp test öbür yaklaşımla çalıştırılır. Saydam kutu ve kara kutu testinin karışımıdır.
- **Kara kutu test verisi üretme yolları:** gereksinimlere göre, eşdeğer bölümleme (equivalence partitioning), sınır değer analizi (boundary value analysis), neden-sonuç grafı (cause-effect graph).

Detaylı Açıklamalar

Süreç içinde test

Gereksinimler analizden alınır. Tasarım yapılır ve değerlendirilir. Kodlama yapılır ve önce sözdizimi denetlenir. Sonra test aşamasına geçilir. Önce test planı hazırlanır. Ardından modüller, sonra bütün sistem test edilir. En son kullanıcı onay testi yapılır ve sistem kullanıma hazırlanır.

Kodlamada kalite

Kodlama kuralları programlama derslerinde işlendiği için bu derste ayrıntıya girilmedi. Hocanın vurguları:

- Açıklama satırları vakit alıyor gibi görünür, ama bakım aşamasında çok işe yarar.
- Değişkenlere anlamlı adlar verilmelidir.
- Kod verimli olmalıdır.

Kalite faktörleri

- **Doğruluk (correctness):** Yazılım spesifikasyonlara ve müşteri isteklerine ne kadar uyuyor?
- **Güvenilirlik (reliability):** Tasarlanan işlevleri istenen duyarlılıkla yerine getiriyor mu?
- **Verimlilik (efficiency):** Bilgi işlem kaynaklarını ne kadar verimli kullanıyor?
- **Güvenlik (security):** Yetkisiz müdahale engelleniyor mu?
- **Kullanışlılık (usability):** Öğrenmek, kullanmak, girdi hazırlamak ve çıktıyı yorumlamak ne kadar kolay? Buna kullanıcı dostu (user friendly) olmak da denir.
- **Hata bulma kolaylığı, esneklik, test edilebilirlik (testability):** Bakım yapacak kişiler için önemlidir.
- **Taşınabilirlik (portability):** Yazılım başka bir donanıma ya da yazılım ortamına ne kadar kolay aktarılır?
- **Yeniden kullanılabilirlik (reusability):** Yazılımın tamamı ya da bir parçası başka bir uygulamada kullanılabilir mi?
- **Birlikte çalışabilirlik (interoperability):** Başka yazılımlarla veri alışverişi ve birlikte çalışma. Hoca "bağlanabilirlik" yerine bu terimi önerdi.

Yeniden kullanılabilirliği sağlayan tasarım kavramları sınıfta soruldu. Cevaplar:

- Nesneye yönelik yaklaşım.
- Az bağımlılık ve modülerlik.
- Tasarım kalıpları.
- Yazılım kütüphaneleri.

Yeniden kullanım daha az iş gücü demektir. Ayrıca test edilmiş parçalar kullanıldığı için testi de kolaylaştırır.

McCall modeli

McCall modelinde kalite faktörleri doğrudan ölçülmez. Her faktör, ilgili yazılım metriklerinin belirli katsayılarla ağırlıklandırılmış toplamı olarak tahmin edilir:

- $F_q = c_1 m_1 + c_2 m_2 + \cdots + c_n m_n$

Burada F_q kalite faktörü, m_i metrikler, c_i ağırlıklardır. Örneğin güvenilirlik, hata sayısı gibi metriklere bağlıdır. Metrikler yazılım proje yönetimi ve ölçümü dersinde işlenecek.

Kaliteyi sağlamak

- Gereksinim analizini iyi yapmak ve planlı geliştirmek.
- Modüllere ayırırken taşınabilirlik ve birlikte çalışabilirliğe önem vermek.
- Ayrıntılı tasarımda (girdi, çıktı, veritabanı tasarımı) doğruluğa ve kullanıcı gereksinimlerine ağırlık vermek.
- Testte performans ve diğer faktörleri ölçmek.

Hata bulma maliyeti süreç ilerledikçe artar, hata bulma olasılığı ise düşer. Bir basamakta yapılan hata, sonraki basamaklara taşınır. Testten önce iki etkinlik daha vardır: gözden geçirme (review) ve yazılım incelemesi (inspection).

Test stratejisi

Birim testi:

- Kod yazıldıkça her modül test edilir. Geliştirmeyle paralel yürür.
- Modülün arayüzü (girdi ve çıktıların uygunluğu), veri yapısı, değişkenlerin durumu ve veri tipleri, kontrol yapıları, ana yollar, hata yolları ve modül sınırları test edilir.
- Hedef, başlangıçtan sona bütün temel yolların denenmesidir. Bunun için farklı girdilerle farklı veri grupları hazırlanır.
- Hatalar daha çok modül sınırlarında çıktığı için sınır değerlere ağırlık verilir.
- Ağırlıklı saydam kutu testi kullanılır, zaman zaman kara kutu testi de yapılır.

Bütünleştirme testi: Modüller birleştirildiğinde arayüzlerin ve modüller arası ilişkilerin doğru olduğu test edilir.

- **Big bang:** Bütün modüller bir kerede birleştirilip test edilir.
- **Artımlı, aşağıdan yukarı:** Alttaki modüller kümeler (cluster) hâlinde birleştirilerek yukarı çıkılır.
- **Artımlı, yukarıdan aşağı:** Üst modülden başlanıp aşağı inilir.
- **Sandviç:** İki yaklaşım birlikte kullanılır.

Sistem testi: Birim ve bütünleştirme testinden geçen yazılımın gerçek sistem ortamında çalışması test edilir: veritabanı yönetim sistemi, donanım ve işletim sistemi.

- **Güvenlik testi:** Saldırı benzetimleriyle dış müdahaleye karşı koruma test edilir.
- **Stres (dayanıklılık) testi:** Miktar, sıklık ve hacim olarak anormal yük verilir. Örneğin çok kullanıcılı sistemde sınır kullanıcı sayısına ulaşıldığında, bir eksiğinde ve bir fazlasında sistemin davranışı denenir.
- **Performans testi:** Cevap süresi ve çalışma performansı ölçülür. Birim testi sırasında da yapılabilir.

Kabul (onay) testi: Bütünleştirme ve sistem testinden sonra yapılır.

- Girdileri: bütünleşmiş yazılım, gereksinimler ve kullanıcı belgeleri.
- SRS ile karşılaştırılarak bir dizi kara kutu testi uygulanır. Uygulama önceden hazırlanmış test planına göre yürür.
- Konfigürasyon gözden geçirmesi (configuration review) de yapılır. Kullanıcı belgeleri, tasarım belgeleri, test belgeleri ve kayıtlar eksiksiz hazırlanmış mı diye bakılır.
- Alfa ve beta testleri bu aşamadadır.

Test türleri

Saydam kutu testi (white box):

- Modülün iç yapısı bilinerek doğrudan modül üzerinde uygulanır. Hoca “beyaz kutu” yerine “saydam kutu” demeyi tercih etti, çünkü iç yapı görülür.
- Daha çok birim testinde kullanılır.
- Programın bütün olası yollarını denemek mümkün değildir. Bu yüzden temel yollar bulunur.
- **Temel yol testi (basis path testing):** McCabe ölçütüyle modüldeki bağımsız yol sayısı bulunur. Her yolu çalıştıracak test verisi hazırlanır. Her mantıksal karar hem doğru hem yanlış dalıyla denenir.
- **Döngü testi:** Basit, iç içe ve birbirine bağlı döngülerde başlama, ilerleme, artım, sınır ve sonsuz döngü hataları aranır.

Kara kutu testi (black box):

- Kaynak kodu ve iç yapıyı bilmeden, dışarıdan yapılır. İşlevler test edilir.
- Daha çok bütünleştirme ve kabul testinde kullanılır.
- Bulduğu hatalar:
  - Eksik ya da hatalı işlevler.
  - Arayüz hataları.
  - Veri yapısı ve veritabanı erişim hataları.
  - Performans hataları.
  - Başlatma ve sonlandırma hataları.
- Hem geçerli hem hatalı veri gönderilir. Hatalı girdilerde sistemin çökmeden devam edip etmediğine bakılır.

Kara kutu test verisi nasıl üretilir?

- **Gereksinimlere göre:** Daha çok kabul testinde kullanılır. Örnek: öğrenci bilgi sisteminde ders kaydı modülü, farklı öğrenci tipleriyle denenir: bölüm öğrencisi, başka bölümden gelen öğrenci, dışarıdan gelen öğrenci.
- **Eşdeğer bölümleme (equivalence partitioning):** Girdi uzayı, aynı davranışı beklediğimiz sınıflara bölünür. Her sınıftan bir test verisi seçilir. Örnek: 1–100 arası bir girdi için 1–25, 26–56, 57–85, 86–100 grupları. Rastgele seçilen veriler aynı aralığa düşebilir; bölümleme farklı alanların test edilmesini sağlar.
- **Sınır değer analizi (boundary value analysis):** En küçük, en büyük ve beklenmeyen uç değerler seçilir. Stres testleri de bu mantıkla yapılır.
- **Neden-sonuç grafı (cause-effect graph):** Adı geçti, ayrıntısına girilmedi.

Gri kutu testi (gray box): İki yaklaşımın karışımıdır. Örneğin test verisi kara kutu gibi hazırlanır ve test saydam kutu gibi çalıştırılır, ya da tersi. Hoca son yıllarda daha çok kullanıldığını söyledi.

Doğrulama ve geçerleme (V&V)

- **Doğrulama (verification):** İşi doğru yolla mı yapıyoruz? Algoritma adımları doğru mu, modüller doğru birleştirilmiş mi?
- **Geçerleme (validation):** Doğru işi mi elde ediyoruz? Ortaya çıkan yazılım müşterinin istediği yazılım mı?
- Birim testinde doğrulama yapılır. Bütünleştirme testinde hem doğrulama hem geçerleme yapılır. Sistem testinde ağırlık geçerlemededir. Kabul testi tamamen geçerlemedir.

Not dağılımı ve sınav biçimi

- **Vize:** Tek vize yapıldı. İkinci vize yok.
- **Not dağılımı:** Notun %60’ı proje ile vize arasında bölünecek. Oranlar projelerin durumuna göre öğrencilerin lehine belirlenecek.
- **Final:** Yüz yüze yapılacak. Değişiklik olursa üniversite duyuracak.

Ders sonrası notlar

- **ISO standardının numarası.** Altyazıda “ISO 9176” diye geçiyor. Söz edilen yazılım kalite modeli büyük olasılıkla **ISO/IEC 9126**’dır. Bu standardın yerini büyük olasılıkla 2011’de **ISO/IEC 25010** aldı.

- **McCall'ın adı.** Altyazıda "MacCol" ve "McColl" diye geçiyor. Doğrusu McCall'dır; model büyük olasılıkla Jim McCall ve arkadaşlarının 1977 tarihli çalışmasına dayanır. McCall modelindeki katsayılı toplam doğrusal bir regresyon denklemine benzer, ama katsayılar kuruma ve projeye göre belirlenir.
- **McCabe'in adı.** Altyazıda "Maccape" ve "makyip" diye geçiyor. Doğrusu Thomas McCabe. Ölçütün adı çevrimsel karmaşıklıktır (cyclomatic complexity): $V(G) = E - N + 2$. Burada E akış grafındaki kenar, N düğüm sayısıdır.
- **Bütünleştirme testinin İngilizcesi.** Hoca bu adımı altyazıda çoğunlukla "bütünlük testi" diye andı; dersin sonunda bir öğrenci "integrity test" dedi, hoca itiraz etmedi. Doğru terim **integration testing**'dir. Integrity, bütünlük ya da veri tutarlılığı anlamına gelir.
- **Sınır değer örneği.** Hoca 200 kullanıcılık bir sistem için yalnız 199 kullanıcıyla denemeyi örnek verdi. Sınır değer analizinde sınırın kendisi (200), bir altı (199) ve bir üstü (201) denenir. Hoca dersin başka bir yerinde bu üç değerin de denenmesi gerektiğini söyledi.
- **Eşdeğer bölümleme örneği.** Örnekteki aralıklar keyfî seçilmiş. Gerçekte bölümler spesifikasyondaki farklı davranışlara göre çıkarılır. Ayrıca geçersiz bölümler de test edilmelidir: 1'den küçük ve 100'den büyük değerler. Altyazıda 85 iki aralıkta birden geçiyor; aralıklar örtüşmemelidir.
- **Döngü testinde "yuvarlanmış" döngü.** Altyazıda "iç içe yuvarlanmış" diye geçen ifade muhtemelen bitişik (concatenated) ya da yapısız (unstructured) döngüleri kastediyor. Standart sınıflama: basit, iç içe (nested), bitişik (concatenated) ve yapısız (unstructured) döngüler.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.